Instituto Superior
de Engenharia
Politécnico de Coimbra

DEPARTAMENTO DE INFORMÁTICA E SISTEMAS

Meta 1 - Análise do problema e desenvolvimento de um modelo com redes neuronais

Relatório de Licenciatura

Autores

Ana Rita Conceição Pessoa – 2023112690

David José Gomes de Matos – 2023143725

INSTITUTO POLITÉCNICO DE COIMBRA

INSTITUTO SUPERIOR DE ENGENHARIA DE COIMBRA

Coimbra, outubro de 2025

1 ÍNDICE

1.1 Índice de texto

1 Índice 1
1.1 Índice de texto 1
2 Descrição do caso de estudo e objetivos do problema 2
3 Descrição do modelo e implementação dos algoritmos 2
3.1 Importação de bibliotecas 2
3.2 Definição dos Diretórios e Parâmetros do Dataset 1
3.3 Carregamento e Pré-processamento das Imagens 1
3.4 Definição e Treino do Modelo 1
3.5 Avaliação do Desempenho 2
4 Análise de resultados 3
4.1 Experiência 1 – 64×64 píxeis, 20 épocas 3
4.2 Experiência 2 - 64×64 píxeis, 30 épocas 4
4.3 Experiência 3 - 32×32 píxeis, 20 épocas 4
5 Conclusões 5
6 Referências 5

2 DESCRIÇÃO DO CASO DE ESTUDO E OBJETIVOS DO PROBLEMA

O projeto aborda o reconhecimento automático de emoções humanas em imagens faciais, classificando-as em cinco categorias: anger (raiva), fear (medo), happy (alegria), sad (tristeza) e surprise (surpresa).
O objetivo é desenvolver um modelo de rede neuronal convolucional (CNN) capaz de identificar corretamente a emoção expressa em cada imagem.

Objetivos:

1. **Classificação precisa:** Treinar uma CNN capaz de distinguir com boa precisão as cinco expressões faciais básicas, analisando o desempenho do modelo em dados sintéticos e reais.
2. **Uso de Dados Reais e Sintéticos:** Utilizar dois conjuntos de dados - *Basic Human Emotions Dataset* (imagens geradas por IA) para treino e validação, e *FER2013* (imagens reais) para teste, de modo a avaliar a capacidade de generalização entre domínios.
3. **Avaliação e Iteração:** Testar diferentes configurações de treino, variando o número de épocas e a resolução das imagens, de forma a observar o seu impacto no desempenho do modelo.
4. **Avaliação de Métricas:** Medir a performance através de *accuracy*, *recall*, *especificidade*, *F1-score* e *AUC (Área sob a Curva ROC)*, de forma a obter uma análise completa do comportamento do classificador.
5. **Aplicação Futura:** Este tipo de modelo pode ser aplicado em sistemas de **interação humano-computador**, **análise comportamental** e **interfaces inteligentes** capazes de reconhecer emoções em tempo real.
6.

3 DESCRIÇÃO DO MODELO E IMPLEMENTAÇÃO DOS ALGORITMOS

O código desenvolvido implementa um sistema de reconhecimento de emoções faciais utilizando redes neuronais convolucionais (CNNs) com recurso à biblioteca TensorFlow/Keras.
O objetivo é classificar automaticamente imagens faciais em cinco emoções: anger, fear, happy, sad e surprise.

3.1 Importação de bibliotecas

```python
# ============================================================
# Importação das bibliotecas e funções necessárias
# ============================================================

# Função que treina o modelo (está no ficheiro train_model.py)
from train_model import train_model

# Métricas do scikit-learn para avaliar os resultados do modelo
# - classification_report -> mostra precisão, recall e f1-score
# - roc_auc_score -> mede a capacidade de distinguir classes (AUC)
# - confusion_matrix -> mostra acertos e erros por classe
from sklearn.metrics import classification_report, roc_auc_score, confusion_matrix

# label_binarize converte os rótulos em formato binário (necessário para AUC)
from sklearn.preprocessing import label_binarize

# Ferramentas do Keras para ler imagens organizadas por pastas
# (normaliza os pixeis e cria lotes automaticamente)
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Para carregar modelos previamente treinados (.h5)
from tensorflow.keras.models import load_model

# Bibliotecas de suporte
# - NumPy: operações matemáticas
# - Matplotlib e Seaborn: geração de gráficos
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Pathlib ajuda a criar caminhos de ficheiros/pastas de forma independente do sistema operativo
from pathlib import Path
```

O código inicia com a importação das principais bibliotecas necessárias para o processamento e treino do modelo:

TensorFlow / Keras – criação e treino da CNN;
Scikit-learn – cálculo das métricas (accuracy, F1-score, AUC, matriz de confusão);
Matplotlib e Seaborn – visualização dos resultados;
NumPy e Pandas – manipulação e análise de dados;
Pathlib e Shutil – organização dos diretórios do dataset.

3.2 Definição dos Diretórios e Parâmetros do Dataset

```
# ============================================================
# 1) Definir as pastas principais do dataset
# ------------------------------------------------------------
# ROOT        → pasta principal do projeto (TP)
# DATASET_DIR → onde estão as imagens
# TRAIN, VAL  → subpastas com as imagens de treino e validação
# POOL        → pasta temporária onde as imagens são reunidas
# ============================================================
ROOT        = Path(__file__).resolve().parent.parent        # .../TP
DATASET_DIR = ROOT / "dataset"
TRAIN = DATASET_DIR / "train"
VAL   = DATASET_DIR / "validation"
POOL  = DATASET_DIR / "_pool_tmp"   # pasta temporária (é apagada no final)
```

```
# Cria as pastas das classes em train e validation se ainda não existirem
for c in CLASSES_OK:
    (TRAIN / c).mkdir(parents=True, exist_ok=True)
    (VAL / c).mkdir(parents=True, exist_ok=True)

# Cria também a pasta temporária "_pool_tmp"
POOL.mkdir(parents=True, exist_ok=True)
```

São definidos os caminhos para as pastas de treino, validação e teste, bem como o tamanho das imagens utilizadas (64×64 ou 32×32 píxeis).
O script fix_split_80_20.py garante a distribuição equilibrada dos dados (80% treino, 20% validação) e reorganiza automaticamente as imagens por classe.

3.3 Carregamento e Pré-processamento das Imagens

```
# ============================================================
# 1. Preparação dos dados
# ------------------------------------------------------------
# As imagens são carregadas com o ImageDataGenerator, que:
# - Lê automaticamente as imagens em cada subpasta (classe)
# - Converte os valores dos píxeis para o intervalo [0, 1]
# ============================================================

train_datagen = ImageDataGenerator(rescale=1./255)   # Normaliza as imagens de treino
val_datagen   = ImageDataGenerator(rescale=1./255)   # Normaliza as imagens de validação

# Criação dos "geradores" que leem as imagens em lotes (batches)
train_generator = train_datagen.flow_from_directory(
    train_dir,                      # Caminho das imagens de treino
    target_size=(img_size, img_size),   # Redimensiona as imagens (64x64 ou 32x32)
    batch_size=batch_size,              # Número de imagens por lote
    class_mode='categorical',           # As classes são categóricas (emoções)
    shuffle=True                        # Baralha as imagens a cada época
)
print(f"Imagens de treino carregadas: {train_generator.samples} imagens")

# O gerador de validação lê as imagens sem baralhar
val_generator = val_datagen.flow_from_directory(
    val_dir,
    target_size=(img_size, img_size),
    batch_size=batch_size,
    class_mode='categorical',
    shuffle=False
)
```

As imagens são carregadas e normalizadas para valores entre 0 e 1, utilizando ImageDataGenerator.
Este processo permite ao modelo treinar de forma mais estável e evitar problemas com escalas diferentes de intensidade de pixels.

3.4 Definição e Treino do Modelo

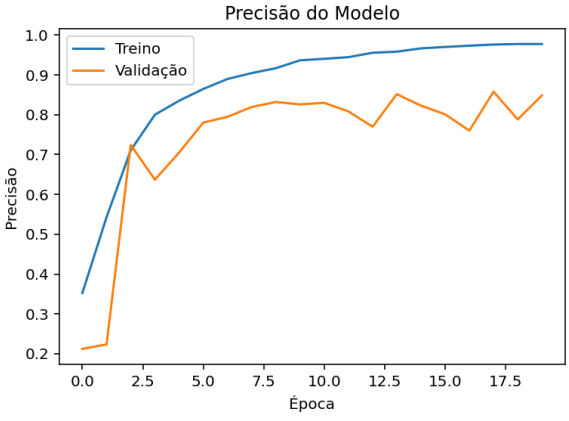
```
# ============================================================
# --- Modelo mais avançado  ---
# Tem 5 blocos de convolução + pooling para aprender padrões
# cada vez mais complexos nas expressões faciais.
# ============================================================
model = Sequential()
model.add(Input(shape=input_shape))  # define tamanho das imagens de entrada

# 1º bloco: aprende padrões básicos (bordas, formas simples)
model.add(Conv2D(32, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

# 2º bloco: combina padrões do anterior (ex: olhos, boca)
model.add(Conv2D(64, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))

# 3º bloco: aprende detalhes mais abstratos
model.add(Conv2D(128, (3,3), activation='relu', padding='same'))
model.add(BatchNormalization())
model.add(MaxPooling2D((2,2)))
```

O ficheiro cnn_model.py define a arquitetura da CNN, composta por camadas convolucionais, *batch normalization*, *max pooling*, *dropout* e camadas densas de saída com ativação *softmax*.
O treino é realizado através do script train_model.py, que executa o método *fit()* e guarda os gráficos de *accuracy* e *loss* por época.



3.5 Avaliação do Desempenho

A avaliação é conduzida pelo ficheiro **main.py**, que testa o modelo no conjunto **FER2013** (imagens reais) e gera:

- Matriz de confusão;
- Relatório de classificação (precisão, sensibilidade, F1-score);
- Valores de AUC por classe.

Estas métricas permitem comparar o desempenho do modelo nas diferentes configurações testadas (número de épocas e resolução das imagens).

```python
# ============================================================
# 2.2 Relatório de Classificação (Precision, Recall, F1-score)
# ------------------------------------------------------------
# Esta métrica mostra o desempenho do modelo para cada emoção.
# Também inclui médias macro e ponderadas (importante para o relatório).
# ============================================================
report = classification_report(
    y_true, y_pred, target_names=class_labels, digits=4
)
print("\n--- Classification report ---")
print(report)

# Guarda o relatório num ficheiro de texto (.txt)
with open(exp_dir / "classification_report.txt", "w", encoding="utf-8") as f:
    f.write(report)

# ============================================================
# 2.3 Cálculo da AUC (Área sob a curva ROC)
# ------------------------------------------------------------
# Mede a capacidade do modelo distinguir cada emoção das restantes.
# A AUC macro é a média global (valor máximo = 1 → ótimo).
# ============================================================
y_true_bin = label_binarize(y_true, classes=list(range(num_classes)))

# Calcula a AUC individual de cada classe
auc_per_class = {
    class_labels[i]: roc_auc_score(y_true_bin[:, i], preds[:, i])
    for i in range(num_classes)
}

# Calcula a AUC média (macro)
auc_macro = roc_auc_score(
    y_true_bin, preds, average='macro', multi_class='ovr'
)

# Mostra os resultados no terminal
print("\nAUC por classe:")
for lbl, auc in auc_per_class.items():
    print(f"{lbl}: {auc:.4f}")
print(f"AUC macro (OvR): {auc_macro:.4f}")

# Guarda os valores de AUC num ficheiro de texto
with open(exp_dir / "auc.txt", "w", encoding="utf-8") as f:
    for lbl, auc in auc_per_class.items():
        f.write(f"{lbl}: {auc:.4f}\n")
    f.write(f"Macro: {auc_macro:.4f}\n")
```

```python
# ============================================================
# 2.4 Matriz de Confusão + Sensibilidade/Especificidade
# ------------------------------------------------------------
# A matriz de confusão mostra onde o modelo acerta e erra.
# A sensibilidade (recall) e especificidade ajudam a perceber
# se o modelo reconhece bem cada emoção.
# ============================================================
cm = confusion_matrix(y_true, y_pred)

# Guarda os valores em ficheiro para análise posterior
with open(exp_dir / "sens_esp.txt", "w", encoding="utf-8") as f:
    f.write("--- Sensibilidade (Recall) e Especificidade ---\n")
    for i, lbl in enumerate(class_labels):
        TP = cm[i, i]  # verdadeiros positivos
        FN = cm[i, :].sum() - TP  # falsos negativos
        FP = cm[:, i].sum() - TP  # falsos positivos
        TN = cm.sum() - (TP + FN + FP)  # verdadeiros negativos

        # Cálculo das métricas
        sens = TP / (TP + FN) if (TP + FN) > 0 else 0.0
        esp  = TN / (TN + FP) if (TN + FP) > 0 else 0.0

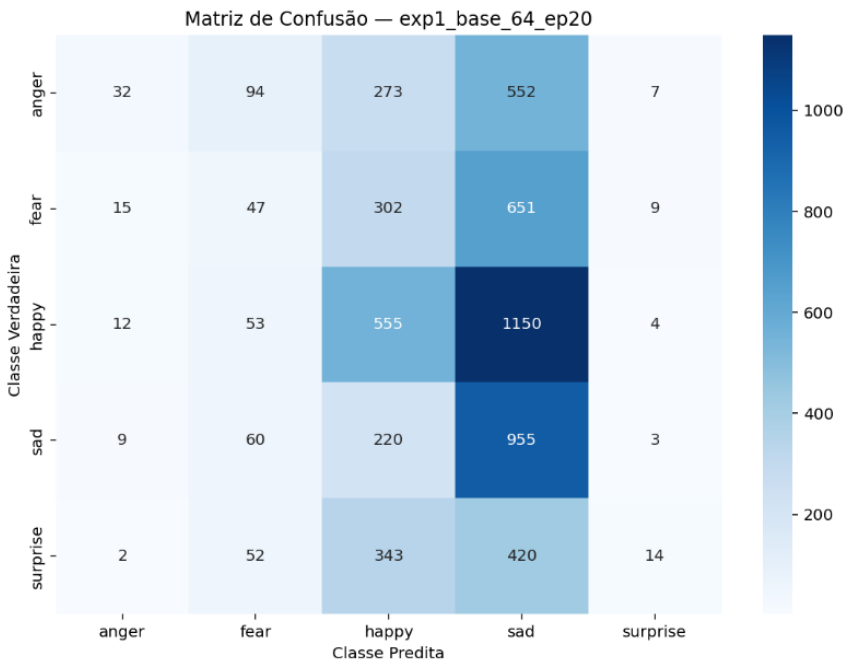
        f.write(f"{lbl:>9s} | Sens: {sens:.4f} | Esp: {esp:.4f}\n")
```

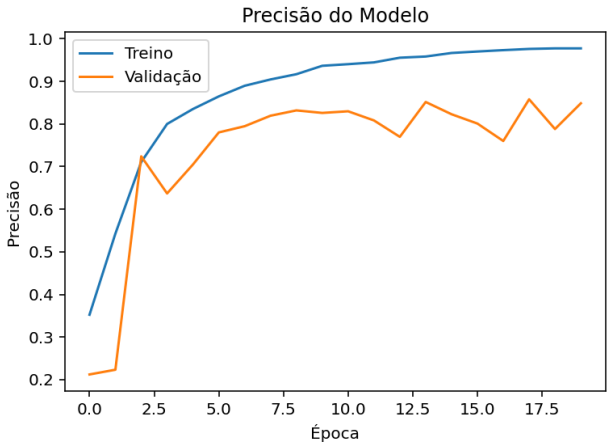
4 ANÁLISE DE RESULTADOS

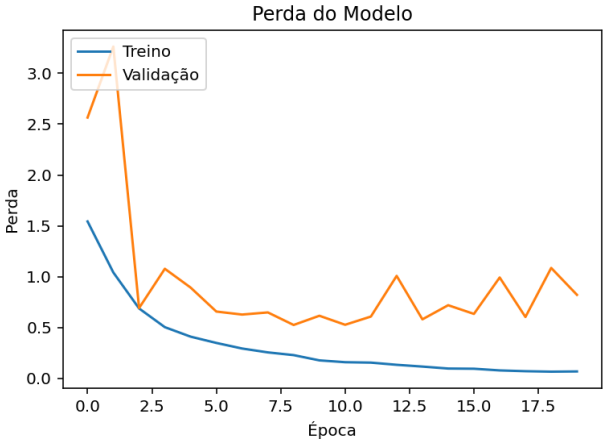
Três experiências foram conduzidas para avaliar o impacto do tamanho da imagem e do número de épocas no desempenho do modelo CNN.
Os resultados incluem os gráficos de *accuracy* e *loss*, a matriz de confusão e a precisão global no conjunto de teste FER2013.

4.1 Experiência 1 – 64×64 píxeis, 20 épocas

Durante o treino, a accuracy atingiu cerca de 97% e a validação estabilizou em 85%, demonstrando boa aprendizagem com início de overfitting.
No conjunto de teste (FER2013), a precisão global foi de 27,5% e o valor médio de AUC = 0.57.
O modelo reconheceu melhor a emoção "sad" (recall 76%, F1 = 0.38), enquanto as emoções "anger", "fear" e "surprise" foram frequentemente confundidas entre si, apresentando valores de recall muito baixos (entre 1% e 5%) e *F1-scores* inferiores a 0.07.
A emoção "happy" teve desempenho intermédio (*recall 31%*, *F1 = 0.32*).
A matriz de confusão mostra um bom reconhecimento de sad e maior dispersão nas restantes classes, indicando dificuldades nas emoções negativas semelhantes.



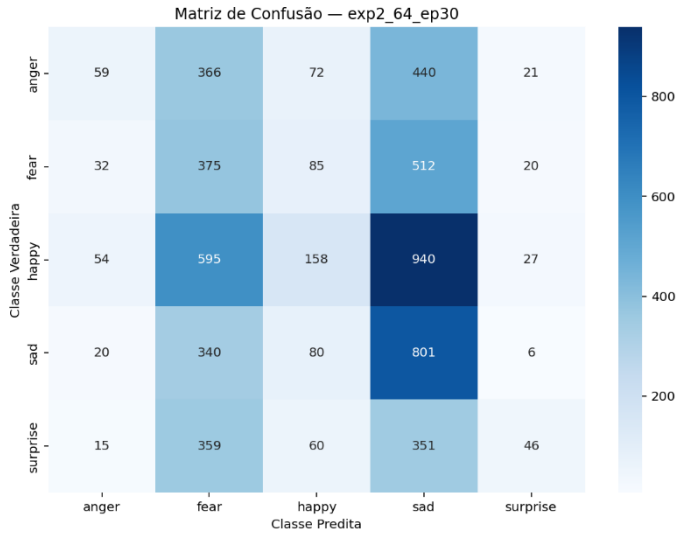


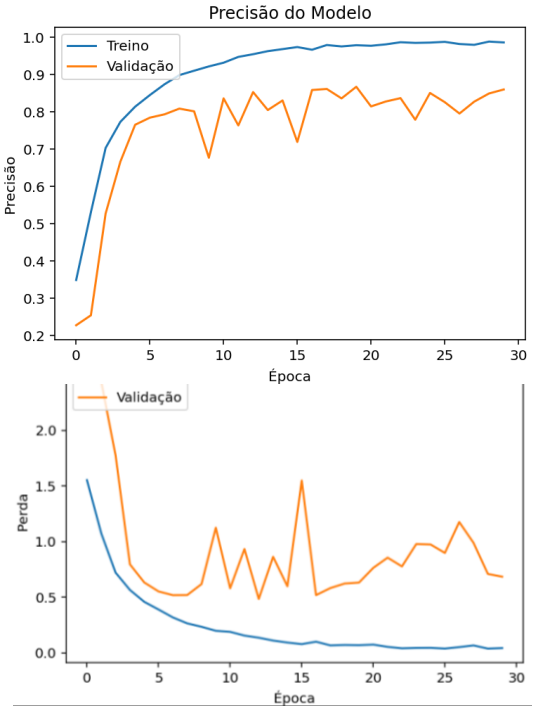


4.2 Experiência 2 - 64×64 píxeis, 30 épocas

Com o aumento para 30 épocas, o modelo atingiu 99% em treino e 83% em validação, mostrando um overfitting mais acentuado.

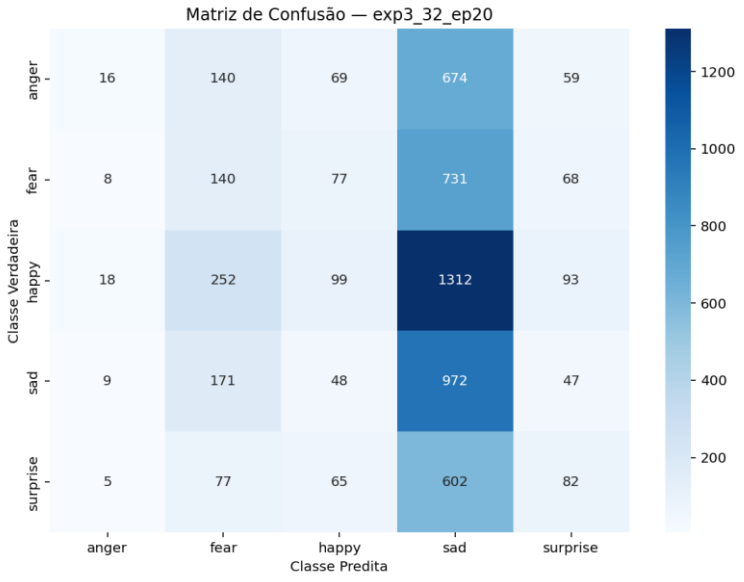
A accuracy no teste desceu para 24,7% e AUC média = 0.56, o que confirma menor capacidade de generalização.
A emoção "sad" manteve o melhor desempenho (recall = 64%, F1 = 0.37), enquanto "fear" obteve um valor intermédio (*recall 37%*, *F1 = 0.25*).
As emoções "anger", "happy" e "surprise" apresentaram *recall* muito baixos (inferiores a 10%) e *F1-scores* abaixo de 0.15.
A matriz de confusão evidencia maior confusão entre emoções negativas, especialmente entre "fear" e "sad", confirmando a perda de generalização do modelo.

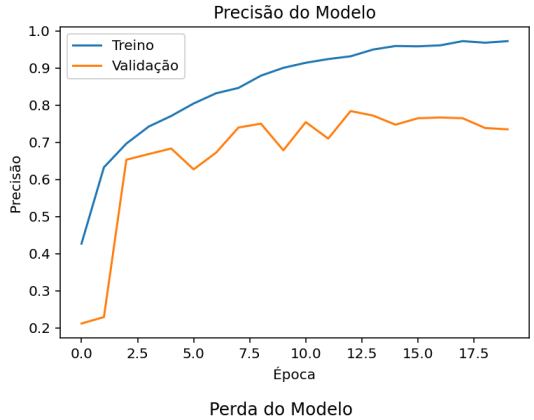


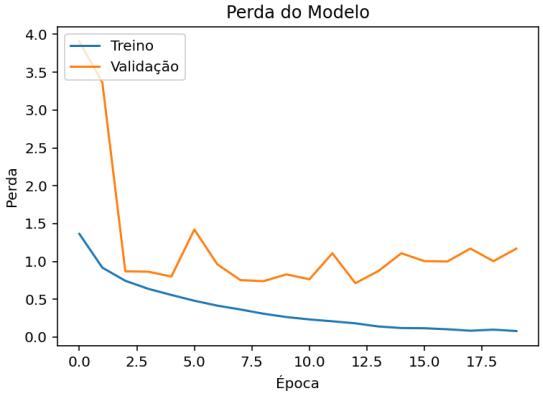


4.3 Experiência 3 - 32×32 píxeis, 20 épocas

Durante o treino, a accuracy atingiu cerca de 92%, enquanto a validação estabilizou em 78%, mostrando uma aprendizagem mais limitada.
No conjunto de teste (FER2013), a precisão global foi de 22,4% e a AUC média de 0.53.
A emoção "sad" manteve o melhor desempenho (recall = 78%, F1=0.35), mas com muitos falsos positivos, enquanto as restantes emoções apresentaram recall inferior a 15% e *F1-scores* abaixo de 0.16.
A matriz de confusão mostra maior concentração de acertos em sad e forte confusão entre emoções negativas, causada pela perda de detalhe facial devido à menor resolução.







5 CONCLUSÕES

Ao comparar as três experiências com variações de resolução e número de épocas no modelo CNN para reconhecimento de emoções, podemos retirar as seguintes conclusões:

- **Variação no número de épocas:** Aumentar de 20 para 30 épocas (Experiência 2) levou a *accuracy* de treino a quase 99 %, mas piorou o desempenho no teste real (24,7 %). Este resultado evidencia que mais épocas amplificam o overfitting, sem ganhos reais de generalização.
- **Variação na resolução:** Reduzir a resolução das imagens de entrada para 32×32 píxeis (Experiência 3) provocou uma diminuição acentuada na precisão (22,4 %), mostrando que a perda de detalhe facial compromete fortemente o reconhecimento emocional.
- **Desempenho por classe:** Em todas as experiências, a emoção "sad" apresentou melhor desempenho (recall entre 64–78 %), enquanto "anger", "fear" e "surprise" tiveram baixas taxas de acerto. O modelo mostrou-se mais sensível a expressões negativas marcadas, mas pouco eficaz em distinguir emoções com padrões subtis.
- **Comparação entre datasets:**

  O modelo foi treinado e validado com o Basic Human Emotions Dataset, composto por imagens geradas por IA com iluminação e enquadramentos uniformes.
  Embora tenha facilitado o treino inicial (permitindo ao modelo atingir elevada *accuracy* interna), a diferença visual entre o dataset sintético (BHE) e o dataset real (FER2013) causou forte perda de generalização.
  Este desfasamento é o principal fator que explica o salto entre as altas *accuracies* de treino (97–99 %) e o baixo desempenho real (22–27 %).
- **Comparação com benchmarks reais:**

  Trabalhos baseados no FER2013 reportam accuracies próximas de 70–75 % com CNNs otimizadas e data augmentation.
  Em comparação, o nosso modelo alcançou 27,5 %, revelando o impacto de um dataset sintético limitado e da ausência de técnicas de regularização e transferência de conhecimento.

Em suma, o desempenho do modelo mostrou-se fortemente dependente da qualidade e do domínio dos dados de treino, confirmando que redes treinadas em bases artificiais dificilmente generalizam bem para imagens reais.

Este trabalho estabeleceu uma base sólida para o reconhecimento de emoções com CNNs, demonstrando que, apesar das limitações de generalização, o modelo é funcional e constitui um ponto de partida robusto para otimizações futuras.

6 REFERÊNCIAS

Datasets

https://www.kaggle.com/datasets/bwandowando/basic-human-emotions

https://www.kaggle.com/datasets/msambare/fer2013

Outros Recursos

https://chat.openai.com/



Instituto Superior
de Engenharia
Politécnico de Coimbra